📜 سیستم حسابداری دوتایی — مستندات پروژه و مسیر توسعه

Full-Stack Double-Entry Accounting System — پایان‌نامه دانشگاهی

> این سند به زبان فارسی، پروژه را به‌طور کامل توضیح می‌دهد: فناوری‌ها، معماری، پایگاه داده، مفاهیم حسابداری، امکانات و مراحل توسعه. مناسب برای ارائه و دفاع پایان‌نامه.

---

۱) معرفی پروژه

یک سیستم حسابداری الکترونیکی یکپارچه که بر اصل **دفترداری دوتایی (Double-Entry Bookkeeping)** ساخته شده است: هر رویداد مالی در دفتر کل با دو سمت ثبت می‌شود — **بدهکار (Debit)** و **بستانکار (Credit)** — و مجموع بدهکار همیشه با مجموع بستانکار برابر است.

ماژول‌ها (صفحات) سیستم:

- داشبورد (Dashboard)
- کدینگ حساب‌ها (Chart of Accounts)
- مشتریان (Customers)
- تأمین‌کنندگان (Suppliers)
- محصولات (Products)
- **خریدها / ورود به انبار (Purchases)**
- فاکتورها و فروش (Invoices)
- پرداخت‌ها (Payments)
- سند روزانه / دفتر کل (Journal)
- گزارش‌های مالی (Reports)
- کاربران (Users)
- گزارش ممیزی (Audit Log)

---

۲) پشته فناوری (Tech Stack)

لایه	فناوری	دلیل انتخاب
فرانت‌اند	**JavaScript** خالص **(Vanilla)** + HTML + CSS — بدون React/Angular	سبک‌تر، شفاف‌تر برای آموزش، منطبق با نیاز پروژه
طراحی	**Tailwind CSS** (سامانه طراحی «Stitch ds») + Material Symbols + فونت Inter	رابط یکدست، پشتیبانی RTL، حالت روشن/تاریک
بک‌اند	**Node.js + Express**	API ساده و سریع
پایگاه داده	**MySQL** با درایور `mysql2`	رابطه‌ای، با پشتیبانی از تراکنش‌های دقیق
احراز هویت	**JWT** + **bcrypt**	نشست‌های امن + هش کردن رمزهای عبور
امنیت	`helmet` + `cors`	هدرهای امنیتی + سیاست مبدأ درخواست
اعتبارسنجی	`express-validator`	عدم اعتماد به ورودی کاربر
پیکربندی	`dotenv` (فایل .env)	مخفی نگه‌داشتن اطلاعات اتصال و کلیدها
حین توسعه	`nodemon`	اجرای خودکار مجدد سرور

کتابخانه‌های واقعی بک‌اند (از `package.json`): `express`، `mysql2`، `bcrypt`، `jsonwebtoken`، `helmet`، `cors`، `dotenv`، `express-validator` و برای توسعه `nodemon`.

---

۳) معماری سیستم (Architecture)

```
[ Frontend  (client/) ]  --HTTP + JSON-->  [ Backend API  (Express :3000) ]  --SQL-->  [ MySQL :3306 ]
   Vanilla JS + fetch                          Routes > Controllers > Services > Models
```

۳.۱ لایه‌های بک‌اند (`server/`)

پوشه	وظیفه
`server.js`	نقطه شروع: نصب میدل‌ورها و ثبت تمام مسیرها
`config/`	`db.js` (اتصال) + `accounting.js` (کدهای پیش‌فرض حساب‌ها)
`routes/`	تعریف آدرس‌های API + سطح دسترسی لازم برای هر مسیر
`controllers/`	لایه نازک HTTP: خواندن درخواست، فراخوانی سرویس، پاسخ JSON یکپارچه
`services/`	**مغز پروژه**: منطق مالی، تراکنش‌ها و قواعد کسب‌وکار
`models/`	تمام کوئری‌های SQL (پارامتریزه‌شده) برای هر جدول
`middleware/`	`auth` (JWT + نقش‌ها)، `errorHandler` ، `validate` متمرکز
`validators/`	قواعد `express-validator`
`utils/`	`money` (محاسبه صحیح به سنت)، `asyncHandler` ، `response` ، `ApiError` ، `i18n` ، `jwt`

الگوی درخواست: `Route → Controller → Service → Model → Database`

۳.۲ ساختار فرانت‌اند (`client/`)

هر صفحه HTML اسکریپت‌ها را با ترتیب ثابت بارگذاری می‌کند:

```
config → api → auth → ui → layout → crud → lookups → <منطق صفحه>
```

فایل	وظیفه
`api.js`	پوشش روی `fetch` : افزودن توکن + هدر زبان `X-App-Lang` و یکپارچه‌سازی خواندن پاسخ
`auth.js`	ذخیره نشست، محافظت از صفحات (`requireAuth`)، خروج
`ui.js`	ترجمه و جهت: `isRtl` ، `bi(en,ar,fa)` ، قالب‌بندی پول `$` ، پوسته
`layout.js`	قالب مشترک (منوی کناری + نوار بالا) با map `NAV`
`crud.js`	ابزارهای مشترک: `renderTable` و `openFormModal`
`lookups.js`	بارگذاری کش‌شده فهرست‌ها (مشتری/محصول/تأمین‌کننده/حساب)

---

۴) پایگاه داده (`database/`)

دو فایل: `schema.sql` (جداول) + `seed.sql` (داده‌های اولیه). اجرا از طریق `scripts/load-db.ps1` .

مهم‌ترین جداول: `chart_of_accounts` · `users` (درخت حساب‌ها) · `products` · `suppliers` · `customers` · `journal_entries` + `journal_lines` · `audit_logs` · `purchases` + `purchase_items` · `invoices` + `invoice_items` (+ `stock_quantity`).

قواعد طراحی مهم:

- **مبالغ از نوع** FLOAT/DOUBLE — `DECIMAL(15,2)` ممنوع (دقت حسابداری).
- **کلیدهای خارجی (FK)** با رفتار حذف محاسبه‌شده:
  - `CASCADE` برای ردیف‌های فرعی (invoice_items/purchase_items).
  - `RESTRICT` برای محافظت از سوابق مالی (حذف مشتری با فاکتور ممکن نیست).
  - `SET NULL` برای پیوندهای حساب.
- **حذف منطقی (Soft Delete)** با `is_active` به‌جای حذف فیزیکی.
- **ستون** `products.inventory_account_id` کالا را از خدمت جدا می‌کند: کالا (دارای حساب انبار → با تعداد پیگیری می‌شود) در برابر خدمت (NULL → فروش بدون انبار).

---

۵) مفاهیم پایه حسابداری و فنی

1. **سند دوتایی** (`journalService`): سند فقط در صورتی ثبت می‌شود که: حداقل ۲ سطر، هر سطر بدهکار یا بستانکار (نه هر دو)، **مجموع بدهکار = مجموع بستانکار**، و هر حساب موجود و برگ (leaf) باشد.
2. **دقت مالی** (`money.js`): همه محاسبات **به‌صورت عدد صحیح (سنت)** انجام می‌شود تا خطای اعشار شناور (`0.1+0.2`) از بین برود؛ تبدیل به «123.45» فقط هنگام نمایش/ذخیره.
3. **تراکنش‌های اتمی (Transactions):** فاکتور + سند آن + حرکت انبار در یک `transaction` واحد (`beginTransaction/commit/rollback`) با قفل‌های `SELECT ... FOR UPDATE` — یا همه موفق می‌شوند یا همه برگشت می‌خورند.
4. **سطوح دسترسی (RBAC):** `viewer` / `accountant` / `admin` . خواندن برای هر کاربر واردشده، نوشتن برای admin+accountant، مدیریت کاربران و ممیزی فقط برای admin.
5. **گزارش ممیزی (Audit):** هر عمل نوشتاری ثبت می‌شود (`auditService`).
6. **یکپارچگی پاسخ:** همه پاسخ‌های JSON ساختار یکسان دارند (`success/message/data/meta` و خطاها `errors[]`).
7. **چندزبانگی و جهت:** سه زبان (EN/AR/FA) با RTL؛ هر عبارت ترجمه‌نشده به‌صورت خودکار به انگلیسی برمی‌گردد. واحد پول **دلار $** در کل سیستم یکسان است.

---

۶) چرخه کار مالی (مثله سوپرمارکت)

1. **خرید از تأمین‌کننده** (صفحه خریدها) ← تعداد افزایش می‌یابد + سند: `بدهکار انبار / بستانکار حساب تأمین‌کننده` .

2. **فاکتور فروش** (پیش‌نویس ← صادرشده) ← هنگام صدور: `بدهکار حساب مشتری / بستانکار درآمد` + **کاهش تعداد** + `بستانکار انبار / (COGS) بدهکار بهای تمام‌شده کالای فروش‌رفته`.
3. **پرداخت از مشتری** ← `بدهکار صندوق/بانک / بستانکار حساب مشتری`.
4. **گزارش‌ها** (تراز آزمایشی، صورت سود و زیان، ترازنامه) ← از اسناد تولید می‌شوند.

> **پیش‌نویس در برابر صادرشده:** پیش‌نویس هیچ اثری بر دفتر کل و انبار ندارد. تبدیل به «صادرشده» است که سند می‌زند و تعداد را کم می‌کند.

---

۷) مسیر توسعه — مراحلی که طی کردیم

> نکته: گفت‌وگوی اصلی طولانی بود و قدیمی‌ترین بخش‌ها خلاصه شده‌اند. مهم‌ترین مراحل به ترتیب:

مرحله ۱ — پایه Full-Stack

- ساختار پروژه (client/server/database)، اتصال به MySQL، میدل‌ورها و خطاهای یکپارچه.
- احراز هویت bcrypt + JWT + نقش‌ها (RBAC) + گزارش ممیزی.
- عملیات CRUD کامل: کدینگ حساب‌ها، مشتریان، تأمین‌کنندگان، محصولات، فاکتورها، پرداخت‌ها، اسناد، گزارش‌ها، کاربران.
- دقت مالی به سنت + سند دوتایی متوازن.

مرحله ۲ — بومی‌سازی و زبان‌ها

- **افزودن فارسی (FA)** به‌عنوان زبان سوم RTL با بازگشت به انگلیسی.
- **یکسان‌سازی واحد پول بر دلار $** در همه صفحات — فقط تغییر نماد/برچسب بدون نرخ تبدیل.
- یکپارچه‌سازی جهت رابط (RTL/LTR) با منطق CSS بدون انشعاب برای هر زبان.

مرحله ۳ — اصلاح حساب‌های بدهکار/بستانکار (تصمیم بنیادی)

- مشکل: مشتری جدید بدون `account_id` ← صدور فاکتور خطا می‌داد.
- راه‌حل: **ایجاد خودکار حساب فرعی** برای هر مشتری (حساب‌های دریافتنی AR زیر ۱۳۰) و هر تأمین‌کننده (حساب‌های پرداختنی AP زیر ۲۱۰۰) در همان تراکنش + اسکریپت `backfillPartyAccounts.js` برای موارد قدیمی.

مرحله ۴ — محدود کردن روش‌های پرداخت

- پرداخت‌ها فقط **نقدی (Cash)** و **کارت (Card)** (حذف بانک/چک).

مرحله ۵ — چرخه عمر فاکتور

- افزودن **صدور (Issue)** برای پیش‌نویس (تبدیل به صادرشده + سند + کاهش انبار) و **حذف (Delete)** فقط برای پیش‌نویس.

- محافظت: فاکتور صادرشده حذف نمی‌شود (باید ابطال شود) و پیش‌نویس سند نمی‌زند.

مرحله ۶ — موجودی انبار (سطح کامل B)

- ستون `stock_quantity` + جداول `purchases` / `purchase_items` + ماژول کامل خرید (API + صفحه).
- خرید تعداد را زیاد می‌کند و سند `بدهکار انبار / بستانکار AP` می‌زند و هزینه آخرین کالا را به‌روز می‌کند.
- فروش تعداد را کم می‌کند و سند `بدهکار COGS / بستانکار انبار` می‌زند و **فروش بیش از موجودی ممنوع** است.
- فایل (idempotent) `database/migrations/001_inventory.sql` برای پایگاه‌های داده موجود.

مرحله ۷ — اصلاح خدمت در برابر کالا

- خطا: فاکتور دارای «خدمت نصب» رد می‌شد (Insufficient stock). علت: سیستم همه محصولات را کالا می‌شمرد.
- راه‌حل: **محصولی که حساب انبار ندارد = خدمت** ← بدون تعداد و بدون COGS فروخته می‌شود و خرید آن برای انبار رد می‌گردد.

مرحله ۸ — اصلاح تولید شماره اسناد

- خطای قدیمی: شماره فاکتور با `COUNT+1` ← برخورد (۴۰۹) هنگامی که حذف، فاصله در ترتیب ایجاد می‌کرد.
- راه‌حل: `MAX(شماره ترتیبی)+1` برای فاکتورها و خریدها (امن در برابر فاصله‌ها).

---

۸) تصمیمات مهندسی و درس‌های آموخته‌شده

- **محاسبات داخل یک تراکنش** — فاکتور بدون سندش (و بالعکس) باقی نمی‌ماند.
- **عدد صحیح (سنت) به‌جای float** — پایه دقت حسابداری.
- **سرور مبدأ حقیقت قیمت است** — قیمت واحد از کاتالوگ محصول خوانده می‌شود، نه از کلاینت.
- **تفکیک کالا از خدمت** با `inventory_account_id` (بدون اختلاط در انبار).
- **تولید شماره بر پایه MAX نه COUNT** (فاصله‌ها باعث برخورد می‌شوند).
- **UTF-8 هنگام وارد کردن SQL عربی/فارسی** با `Start-Process -RedirectStandardInput` (پایپ متن را خراب می‌کند).

---

۹) نحوه اجرای پروژه به‌صورت محلی

```
# ۱) پایگاه داده (XAMPP MySQL روی 3306:)
npm run db:load          # اول schema.sql سپس seed.sql

# ۲) سرور API روی 3000:
npm start                # یا: npm run dev (nodemon)

# ۳) رابط کاربری روی 5500: (سرور فایل‌های ثابت)
npx http-server client -p 5500 -c-1
```

- **ورود:** `admin / Admin@123` (همچنین `viewer` / `accountant`).
- **محیط:** فایل `.env` شامل اطلاعات اتصال و `JWT_SECRET` (خارج از git).

---

۱۰) نکات برای ارائه پایان‌نامه

- **سند دوتایی** را با یک مثال زنده توضیح دهید: فاکتور ← چگونه بدهکار/بستانکار متوازن شد.
- **دقت مالی** (سنت، DECIMAL) را به‌عنوان نقطه قوت نشان دهید.
- **RBAC** را نشان دهید: یک سیستم با سه سطح دسترسی و رابط‌های متفاوت.
- **انبار** را نشان دهید: خرید افزایش می‌دهد، فروش کاهش می‌دهد، جلوگیری از کسری، و COGS.
- **چندزبانگی + RTL** را به‌عنوان یک ویژگی پیشرفته رابط معرفی کنید.

این سند از گفت‌وگوی توسعه تولید شده است. نسخه خام جلسه در `docs/chat-history/` قرار دارد. برای مشاهده نسخه عربی به `docs/Project-Journey.md` مراجعه کنید.